

Selenium Adaptability Report

DESAFIO DE AUTOMAÇÃO: SELENIUM-BOOKS-SCRAPER

1. Environment Setup

Para este relatório, configurei um novo ambiente de automação utilizando **TypeScript** sobre o runtime do Node.js, focando em tipagem estática para evitar erros comuns em tempo de execução.

- **Linguagem:** TypeScript
- **Browser:** Google Chrome (versão estável)
- **Ferramentas Instaladas:**
 - Node.js v18+
 - NPM (Gerenciador de pacotes)
 - Selenium WebDriver Library
 - ts-node (para execução direta de TS)
 - ChromeDriver

2. The Converted Script

O script abaixo realiza o scraping de dados (títulos e preços) do site *Books to Scrape*, utilizando as melhores práticas de espera explícita.

```
import { Builder, By, until, WebDriver } from 'selenium-webdriver';
import * as chrome from 'selenium-webdriver/chrome';

async function scrapeBooks() {
  // Configuração do Driver para Chrome
  let driver: WebDriver = await new Builder()
    .forBrowser('chrome')
    .setChromeOptions(new chrome.Options())
    .build();

  try {
    // Navegação para o site alvo
    await driver.get('https://books.toscrape.com/');

    // Espera explícita para garantir que os elementos carregaram
    await driver.wait(until.elementLocated(By.className('product_pod')), 10000);

    const books = await driver.findElements(By.className('product_pod'));
    console.log(`[LOG] Sucesso: Encontrados ${books.length} livros.`);

    for (let book of books) {
```

```

        const title = await book.findElement(By.css('h3
a')).getAttribute('title');
        const price = await book.findElement(By.css('.price_color')).getText();
        console.log(`- Item: ${title} | Preço: ${price}`);
    }
} catch (error) {
    console.error('[ERRO] Falha na execução:', error);
} finally {
    // Encerramento da sessão
    await driver.quit();
}
}

scrapeBooks();

```

3. Reflection (The "Aha!" Moment)

A Nuance Identificada: Ao migrar de ambientes mais simples para o TypeScript, o meu momento de clareza foi entender a rigidez das **Promises** no Selenium.

Diferente de scripts síncronos, cada interação com o browser retorna uma promessa que *deve* ser resolvida. A "nuance" que mais me chamou a atenção foi a necessidade de tipar o WebDriver corretamente para ter acesso ao IntelliSense, o que torna o trabalho do QA muito mais produtivo e menos propenso a erros de digitação em seletores ou métodos.

4. Proof of Execution

Abaixo, a saída do terminal demonstrando a execução bem-sucedida do script e a extração dos dados:

```

> selenium-books-scraper@1.0.0 start
> ts-node index.ts

[LOG] Sucesso: Encontrados 20 livros.
- Item: A Light in the Attic | Preço: £51.77
- Item: Tipping the Velvet | Preço: £53.74
- Item: Soumission | Preço: £50.10
- Item: Sharp Objects | Preço: £47.82
- Item: Sapiens: A Brief History of Humankind | Preço: £54.23
...
Process finished with exit code 0

```

Este documento foi gerado como parte dos requisitos de conclusão do curso de QA.